

应对 CAT 中的选择偏差：用户级聚合影响函数 (UserAIF) 精读笔记

原文: 3583780.3615455.pdf (CIKM '23, Riiid AI Research) · 代码: <https://github.com/riiid/UserAIF> (已 clone 到本目录 UserAIF/) · 笔记于 2026-06-27

一、解决什么问题

痛点：CAT 启动太贵。计算机自适应测验 (CAT) 要工作，前提是先有一批高质量的**题目画像** (item profile, 即每道题的难度、区分度等参数)。传统做法是先攒一份稠密、无偏的题目一作答数据 (让大量学生把每道题都做一遍)，再训练诊断模型 (如 IRT) 得到题目参数。这个「冷启动」数据采集既花钱又花时间。

一个诱人的想法：能不能边服务边学？CAT 服务一上线，每天都有真实学生在答题、产生新数据。如果能直接拿这些 CAT 服务数据去持续更新题目画像，就能从一个小的初始数据集起步，省掉大规模冷启动。

拦路虎：选择偏差 (selection bias)。CAT 的本质就是「给强的学生出难题、给弱的学生出简单题」(题目选择算法专挑当前作答概率接近 0.5、信息量最大的题)。结果是：

- 难题主要被强者作答 → 难题的正确率被「抬高」→ 诊断模型误以为难题没那么难；
- CAT 数据里题目难度和用户能力被人为地强绑定。

作者在真实数据 (R.test 的 SAT 阅读) 上量化了这个问题：用大规模无偏数据估的题目难度，和用 CAT 有偏数据估的题目难度，二者秩相关只有 **0.0881** (几乎不相关)。直接把 CAT 数据喂进模型，题目参数会严重失真。

和已有工作比好在哪：- 推荐系统里也有选择偏差，常用 **IPS** (逆倾向得分加权) 去偏，但 IPS 要估倾向得分，在 CAT 这种稀疏数据上估不准。- 经典影响函数 (**IF**) 去偏 (Koh & Liang 2017, IF4URec) 是逐个数据点判断「这条数据该不该要」，忽略了一组数据之间的交叉效应，而且依赖一个额外的小验证集。- 本文是**首个用真实 CAT 服务数据研究该偏差的工作**，并提出**按用户聚合的 IF 选择法 UserAIF**，简单、不需要额外验证集、还能保住每个用户内部的题目难度排序信息。

二、问题定义 (输入 / 输出)

两批数据：

- **无偏集 x** ：初始稠密数据。用户集 $\mathcal{U}_{unbiased}$ ，题目集 $\mathcal{Q}$ ，每个元素 $x_{ij} \in \{0, 1\}$ 表示用户 i 答题 j 对/错。每个无偏用户几乎把所有题都做了 ($Q_i = \mathcal{Q}$)。
- **有偏集 z** ：CAT 服务收集的数据。用户集 $\mathcal{U}_{biased}$ ，每个用户 k 只做了 CAT 给他个性化挑的一小串题 $\mathcal{Q}_k$ ， $z_{kj} \in \{0, 1\}$ 。

模型 (诊断模型，这里用 2 参数 IRT)：用户能力嵌入 $\theta_i \in \mathbb{R}^{1 \times m}$ ，题目嵌入 $d_j \in \mathbb{R}^{1 \times n}$ ，预测答对概率 $p_{ij} = \mathbb{P}(x_{ij} = 1 | d_j, \theta_i)$ 。

任务输入：无偏集 x (小) + 有偏 CAT 集 z (带选择偏差)。**任务输出**：一组从 z 中筛选出的子集 $\{z_k\}_{k \in \mathcal{U}_{biased}^s}$ ，使得用 $x \cup$ 这个子集重训练出的题目画像，既比只用 x 更准 (因为多了真实数据)，又不会被偏差带歪 (题目难度排序还接近真实值)。

一个具体小例子：> 初始无偏集：40 个学生把 185 道 ENEM 题都做了 (这就是 x)。CAT 上线后，来了 1000 个新用户，每人被自适应地施测了 30 道题 (这就是 z ， $|\mathcal{U}_{biased}| = 1000$)。其中「学霸用户 A」被专挑了 30 道难题且大多答对——这条用户的数据会把难题难度估偏，应该被剔除；「中等用户 B」拿到的题难度分布比较均衡——它的数据是「干净的增量」，应该保留。UserAIF 要做的就是自动把 A 这类用户筛掉、把 B 这类用户留下。

三、模型结构 (方法分三步走)

整个流程不是一个端到端网络，而是「训练 → 算影响 → 按用户聚合筛选 → 重训练」的数据选择管线：

1. 在无偏集上训练基础模型：用式 (2) 在 x 上训出 IRT 参数 $\hat{\beta}$ (所有题目嵌入 + 无偏用户嵌入)。这是「干净的参考系」。
2. 算每条有偏交互的影响函数 **IF**：对 z 里每条 z_{kj} ，用式 (4) 算它若被加进训练集会把参数 $\hat{\beta}$ 推动多少 (这一步要海森矩阵求逆)。
3. 按用户聚合成 **AIF** (核心创新)：把同一用户 k 的所有交互、所有参数维度的 IF 全加起来，得到这个用户的「总扰动量」 $\text{AIF}_{\beta}(k, x)$ (式 6)。
4. 按用户筛选 + 重训练：取 $|\text{AIF}|$ 小 (扰动小 = 偏差小) 的那部分用户，和无偏集一起重训练 IRT。

为什么要「按用户」而不是「按交互」？两个理由：- **保住排序信息**：一个用户做的题里隐含了「这几道题对他而言谁难谁易」的相对排序。如果按单条交互挑挑拣拣，会把这个用户的题目序列拆碎，秩信息就丢了。整段保留才能维持难度排序。- **考虑组效应**：单条数据的 IF 是线性近似，一组数据一起加入时有交叉效应 ([Koh 2019])。把一个用户的数据当一个「组」来看，更接近真实影响。

四、关键公式详解

公式 (1): 单点交叉熵损失

$$l(x_{ij} | d_j, \theta_i) = -x_{ij} \log p_{ij} - (1 - x_{ij}) \log(1 - p_{ij}).$$

- 符号: p_{ij} 是模型预测的答对概率 (IRT 里 $p_{ij} = \sigma(a_j \theta_i - d_j)$, a_j 区分度、 d_j 难度)。
- 举例: 某题真实标签 $x_{ij} = 1$ (答对), 模型给 $p_{ij} = 0.8$, 则 $l = -\log 0.8 \approx 0.22$, 损失小; 若模型只给 $p_{ij} = 0.2$, 则 $l = -\log 0.2 \approx 1.61$, 损失大。这就是普通的逻辑回归损失。
- 作用: 训练诊断模型的基本砖块; 下面所有梯度、海森都从它来。

公式 (2): 无偏集上的总损失 $\rightarrow$ 得到参考参数 $\hat{\beta}$

$$L(x | \beta) = \frac{1}{\sum_i |\mathcal{Q}_i|} \sum_{i \in \mathcal{U}_{unbiased}} \sum_{j \in \mathcal{Q}_i} l(x_{ij} | d_j, \theta_i).$$

- 就是把所有无偏交互的损失平均。最小化它得到 $\hat{\beta} = \arg \min_{\beta} L$, 即“干净参考系”的参数。 $\hat{\beta}$ 把所有题目嵌入和所有无偏用户嵌入拼成一个长向量, 维度 $n|\mathcal{Q}| + m|\mathcal{U}_{unbiased}|$ 。

公式 (3)(4): 影响函数 IF ——一条新数据会把参数推多远 (重点)

定义 (式 3): 把一条有偏数据 z_{kj} 以权重 ϵ 掺进训练集, 看参数的变化率:

$$\text{IF}_{\beta}(z_{kj}, x) = \lim_{\epsilon \rightarrow 0} \frac{\hat{\beta}_{\epsilon, z_{kj}} - \hat{\beta}}{\epsilon}.$$

闭式解 (式 4, 经典 Hampel / Koh-Liang 结果):

$$\text{IF}_{\beta}(z_{kj}, x) = -H_x^{-1} [\nabla_{\hat{\beta}} l(z_{kj})]^T.$$

- 符号: $H_x = \nabla_{\hat{\beta}}^2 L$ 是在 $\hat{\beta}$ 处的海森矩阵 (刻画损失曲面的曲率); $\nabla_{\hat{\beta}} l(z_{kj})$ 是这条新数据损失对参数的梯度。返回的是一个和 β 等长的向量——每个分量告诉你这条数据会把对应那个参数 (某道题的难度、区分度, 或某个用户的能力) 推动多少、往哪个方向。
- 直觉: 梯度告诉你“这条数据想往哪个方向掰参数”, 乘以 $-H^{-1}$ 把它换算成“实际能掰多少” (曲率越平、掰得越多)。
- 关键技巧: 对有偏用户 k , 作者不给他新建一个能力参数去优化, 而是直接用 CAT 现场估好的 θ_k^{biased} 当固定值代入。这样待估参数还是原来那套 β , 海森不用变, 问题大大简化 (测验够长时 θ_k^{biased} 已经够准)。
- 数值小例子: 假设当前题 j 的难度参数 d_j , 一条“学霸答对难题”的数据 $z_{kj} = 1$ 而模型本以为 $p_{kj} = 0.3$ (觉得他多半会错)。梯度 $\partial l / \partial (\text{logit}) = p_{kj} - z_{kj} = 0.3 - 1 = -0.7$, 说明这条数据强烈想把这道题的 logit 推高 (即把难度 d_j 调低、让题“变简单”)。乘上 $-H^{-1}$ 后, IF 在 d_j 分量上就是一个较大的负值——量化了“这条数据会把难题估得偏简单”这件事。

公式 (5): 损失上的 IF (基线 IF4URec / IFloss 用, 本文不直接用作选择核心)

$$\text{IF}_{loss}(z_{kj}, y) = \sum_{p \in \mathcal{U}_{unbiased}^{val}} \sum_{q \in \mathcal{Q}_p} (-\nabla_{\hat{\beta}} l(y_{pq}) H_x^{-1} [\nabla_{\hat{\beta}} l(z_{kj})]^T).$$

- 含义: 加入 z_{kj} 后, 在一个独立无偏验证集 y 上的损失会变好还是变坏。< 0 表示验证损失下降 (这条数据有益)。
- 本文为啥不主推它: 要额外准备验证集 y , 而 CAT 场景里无偏数据本来就少, 分一块出来做验证很奢侈。UserAIF 用式 (4) 的参数级 IF, 不需要验证集。

公式 (6): 用户级聚合影响函数 AIF (全文最核心)

$$\text{AIF}_{\beta}(k, x) = \sum_{j \in \mathcal{Q}_k} \sum_c (\text{IF}_{\beta}(z_{kj}, x))_c.$$

- 两层求和: 内层 $\sum_c$ 把 IF 向量的所有参数分量加起来 (c 从 1 到 β 长度); 外层 $\sum_{j \in \mathcal{Q}_k}$ 把这个用户 k 做过的所有题加起来。结果是一个标量: 用户 k 的全部数据对全体参数造成的“净扰动总量”。

- **举例走一遍**：用户 A 做了 3 道题，假设每道题算出的 IF 向量按分量求和后分别是 +0.5, +0.6, +0.4 (同号，因为学霸答对难题这件事在每道题上都一致地把难度往同一方向推)，那么 $\text{AIF}(A) = 0.5 + 0.6 + 0.4 = 1.5$, $|\text{AIF}| = 1.5$ ，很大 $\rightarrow$ A 偏差大，剔除。另一个用户 B 做的 3 道题 IF 求和是 +0.3, -0.25, -0.1 (有正有负、相互抵消)， $\text{AIF}(B) = -0.05$, $|\text{AIF}| = 0.05$ ，很小 $\rightarrow$ B 的数据整体几乎不辨参数，保留。
- **为什么是“同号才大”**：选择偏差的特征恰恰是**系统性、同方向的扰动**（强者总是把难题往简单方向推）。聚合求和会让这种同向偏差累加放大，而随机噪声会相互抵消。所以 $|\text{AIF}|$ 大 $\approx$ 系统性偏差重。这正是“聚合”比“逐点”高明的地方。
- **作用**：把“该不该要这个用户的数据”压缩成一个可排序的标量。

选择规则 (UserAIF 三步)

$$\mathcal{U}_{biased}^s = \{k \in \mathcal{U}_{biased} : |\text{AIF}_\beta(k, x)| \leq \eta\},$$

阈值 η 取所有用户 $|\text{AIF}|$ 的下四分位数 (**25%**) ——即只保留扰动最小的那 25% 用户。论文说结果对这个阈值的取法不敏感。然后用 $x \cup \{z_k\}_{k \in \mathcal{U}_{biased}^s}$ 重训。

五、代码实现对照

仓库核心文件 UserAIF/IF.py，把上面公式逐行对上：

- **calculate_IF(...)** (21–124 行) = 公式 (4)。
 - 梯度部分(43–49 行): `_grad_L_biased = (b_prob - b_is_correct) * [theta_i, -1]`，这正是 $\nabla_{\beta} l(z_{kj})$ 。注意 $(p - x)$ 就是上面例子里的 $0.3 - 1 = -0.7$ ；乘上 $[\theta, -1]$ 分别对应分区度 a 和难度 d 的偏导（难度前是 -1 ，所以代码里是 `-torch.ones`）。
 - 海森部分 (51–117 行)：手写解析海森的各块 H_{aa} , H_{dd} , H_{ad} (题目参数块)、 H_{tt} (用户能力块)、 H_{at} , H_{dt} (交叉块)，拼成大矩阵后 `hessian_inv = torch.linalg.inv(hessian)`，对应 H_x^{-1} 。
 - 最后 (122 行)：`IF_total = -(hessian_inv @ grad_L_biased.T).T`，就是公式 (4) 的 $-H^{-1}[\nabla l]^T$ ；`biased_df["IF"] = IF_total.sum(1)` (123 行) 已经把每条交互的 IF 向量按参数分量求和了 (公式 6 的内层 $\sum_c$)。
- **UserAIF(...)** (8–17 行) = 公式 (6) + 选择规则。
 - `groupby("user_id").agg("sum")` (10 行)：把同一用户所有交互的 IF 相加 = 公式 (6) 的外层 $\sum_{j \in Q_k}$ 。
 - `abs(...)` (11 行)：取 $|\text{AIF}|$ 。
 - 阈值 (12–14 行)：`describe().loc["25%"]["IF"]` 就是下四分位数 η ；保留 $\text{IF} < 25\%$ 分位的用户。
 - 返回这些用户的全部交互 `selected_df`，交给主流程和无偏集一起重训。
- **IRT 模型** UserAIF/model/irt.py: `forward` (63–87 行) 实现 $p = \sigma(\sum a \cdot \theta - d)$ ，区分度 `a_params` 用 `softplus` 保证非负 (`constraint="non_negativity"`)；`user_update` (98–150 行) 用 `scipy.optimize.minimize` 在固定题目参数下估每个用户能力 θ ——对应论文里“用 CAT 现场估好的 θ_k^{biased} 当固定值”那一步。
- 训练入口 `main_train.py`，数据集类在 `dataset/` (含 `enem_dataset.py`)。

六、小结

实验结论：- 直接拿有偏 CAT 数据重训 IRT，所有指标都崩（难度秩相关低），证实选择偏差的破坏力。- UserAIF 在 ENEM、NIPS-EDU 两个模拟数据集上，CAT AUC 和题目难度秩相关都优于所有基线（重训练、IPS-NB/IW、IF4URec、IFloss、IFparam、GreedyAIF）。- 偏差越强（KLI 比 FI 偏差更大、 $\sum |\text{AIF}|$ 更大），UserAIF 的提升越明显。- 在神经模型 BOBCAT(BiNN-Approx) + 稀疏 Assist2009 上也有效；在真实 R.test SAT 阅读数据上，小步数 (step 3) 时 MAE、AUC 改善尤为明显。

优点：简单、即插即用；不需要额外无偏验证集；按用户聚合天然契合“系统性偏差同向累加”的特性，并保住题内难度排序；对阈值不敏感；对 IRT 和神经诊断模型都适用。

局限：提升幅度有时偏小（尤其当初始无偏数据远大于有偏数据时）；海森求逆在参数量大时成本高（代码靠 IRT 海森的解析稀疏结构硬算）； θ_k^{biased} 固定的近似要求 CAT 测验长度够长；阈值是静态的。作者把“自适应数据选择策略”和“用二阶 IF 更精确刻画组效应”列为未来工作。